

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Title of the master thesis

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc.

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Max Mustermann
Student ID: 111222
Date: 12.12.1212

Supervising tutor: Prof. Dr. W. Hardt
(further supervisors)

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Keywords: Keyword1, Keyword2, Keyword3, ...max 5

Contents

Contents . . . 3

List of Figures . . . 5

List of Tables . . . 7

List of Abbreviations . . . 8

1 Introduction . . . 9

2 Background and State of the Art . . . 11

3 Data Acquisition and Dataset Construction . . . 12
3.1 Sensor Platform and Computational Constraints . . . 12
3.2 Assembly Scenario and Activity Taxonomy . . . 13
3.3 Data-Collection Firmware . . . 15
3.3.1 Dual-Stream UDP Architecture . . . 15
3.3.2 Audio Double-Buffering and Packet Framing . . . 15
3.3.3 Microsecond Timestamping and Sequence Numbering . . . 16
3.3.4 Multi-Device Capture and Body Placements . . . 17
3.4 Recording Protocol . . . 17
3.5 Ground-Truth Annotation from Synchronised Video . . . 18
3.5.1 Determination of the Audio-Video Offset . . . 19
3.5.2 Interval Labels and Sample-Level Label Embedding . . . 19
3.6 Signal Conditioning . . . 20
3.6.1 Packet-Loss Detection and Audio-Dropout Reconstruction . . . 21
3.6.2 IMU Decimation to 80 Hz . . . 21
3.6.3 Audio Resampling to a Strict 16 kHz Timeline . . . 22
3.7 Window Generation and Labelling . . . 22
3.8 Dataset Statistics, Class Distribution, and Imbalance . . . 24
3.9 Data-Quality Assessment and Known Artefacts . . . 25
3.10 Chapter Summary . . . 26

4 Model Design, Training and Deployment . . . 27
4.1 Overview of the Experimental Design . . . 27
4.2 Input Representations . . . 29
4.2.1 Inertial Windows . . . 29

4.2.2 Audio Log-Mel Spectrograms 29
4.3 Single-Modality Models 31
4.3.1 Inertial Model 31
4.3.2 Audio Model and Temporal Pooling Variants 31
4.4 Fusion Architectures 32
4.4.1 Early Fusion 34
4.4.2 Embedding-Level Fusion 34
4.4.3 Decision-Level Fusion 34
4.5 Classical Baseline 36
4.6 Deployment-Oriented Model Variants 37
4.7 Training Procedure 39
4.7.1 Software and Hardware Environment 39
4.7.2 Training Settings 39
4.8 Deployment on the Wearable Device 40
4.8.1 Target Platform and Software Stack 41
4.8.2 Model Conversion and Integer Quantization 41
4.8.3 On-Device Signal Processing 42
4.8.4 Firmware Structure and Inference Loop 44
4.8.5 Verification of the Deployment Pipeline 44
4.9 Evaluation Protocol 45
4.9.1 Grouped Train/Validation Split 45
4.9.2 Metrics 46
4.9.3 On-Device Measurements 47
4.10 Chapter Summary 47

5 Results and Evaluation 49

6 Discussion 50

7 Conclusion 51

List of Figures

3.1 The M5StickC Plus2 wearable device. The ESP32-PICO-V3-02 system-on-chip, the six-axis IMU (MPU6886), and the microphone (SPM1423) are all integrated on the board. Image: M5Stack [**?**]. 13

3.2 Architecture of the multimodal data-acquisition system. Each of the four wearable devices samples an IMU and a microphone independently, timestamps and frames the samples, and streams them as two separate UDP streams (inertial on one port, audio on another) to a single host, which separates the devices by packet source address and writes raw per-device files. 16

3.3 Signal-conditioning and windowing pipeline, applied per device. The inertial and acoustic branches are brought to fixed rates before both are cut on the same window boundaries. 20

3.4 Window generation and majority-vote labelling. Two-second windows advance in one-second steps (W_1, W_2, W_3 shown stacked for clarity). A window takes the majority activity among its labelled samples; a window whose labelled coverage falls below 70 %, such as one spanning the transition region, is discarded. 23

3.5 Number of two-second windows per activity class in the dominant-wrist dataset ($n = 1380$). The distribution is dominated by the screwdriving classes; `screw_tightening` alone accounts for about 38 % of the windows and the two screwdriving classes together for more than half. 25

4.1 Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models. 30

4.2 Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries. 32

4.3 The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models. 33

4.4 Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom). 37
4.5 Export of a trained model to the device firmware. All constants used on the device are generated automatically from the training pipeline. 42
4.6 Structure of the inference firmware. After the start-up phase, recording, preprocessing, inference and output are repeated continuously. . . 45
4.7 Grouped splitting. Overlapping windows are grouped into activity instances, and whole instances are assigned to either the training or the validation set. A random split would distribute the overlapping windows of one instance across both sets. 46

List of Tables

3.1 Activity classes in the modelling dataset. Descriptions are taken from the 32-step assembly protocol. "Typ. dur." is the range of mean coded-segment durations across the three recording sessions; for `screw_tightening` it covers both the tightening and the merged loosening segments. "Win." is the number of two-second windows per class in the final dominant-wrist dataset (Section 3.8). 14
3.2 Body placements: data recorded during acquisition versus data used as model input in the final pipeline. Only the dominant-wrist inertial and acoustic streams reach the model. 18
3.3 Data-acquisition sessions. "Session" paraphrases the folder label used in the project records. "(+1)" denotes one further sequence marked invalid and set aside. 18
3.4 Composition of the dominant-wrist modelling dataset. 24
3.5 Data-quality indicators for the dataset. 25

4.1 Overview of the evaluated models. 28
4.2 Input representations of the two modalities. 30
4.3 Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations. 33
4.4 Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments. 36
4.5 Hand-crafted features of the classical baseline. 38
4.6 Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer. 39
4.7 Training settings shared by all neural network models. 40
4.8 Firmware variants for on-device inference. The tensor arena is the fixed memory block that holds the intermediate results of a network. 41
4.9 Values embedded in the firmware in addition to the model weights. . 43
4.10 Training and validation windows per class after the grouped split. . . 47

List of Abbreviations

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

1 Introduction

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus

tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

2 Background and State of the Art

3 Data Acquisition and Dataset Construction

This chapter explains the process used to create the multimodal dataset used in the remainder of this thesis. It introduces the wearable sensing platform and its resource envelope, the assembly scenario, the activity classes recorded, the firmware that streamed inertial and acoustic data off the devices, the protocol used to record participants, the video-based annotation procedure, and the steps of signal conditioning and windowing that transform the raw streams into labelled two-second segments. The chapter ends with a quantitative characterisation of the resulting dataset and an assessment of its known limitations. The modelling choices made on this dataset are deferred to Chapter 4, and the implications of the dataset's limitations for the validity of the results are addressed in Chapter 6.

3.1 Sensor Platform and Computational Constraints

The sensing platform is the M5StickC Plus2, a self-contained wearable device built around the ESP32-PICO-V3-02 system-on-chip (SoC), a dual-core processor clocked up to 240 MHz [**?**]. Each device is equipped with a six-axis inertial measurement unit (IMU, MPU6886) providing three-axis acceleration and three-axis angular velocity, a digital MEMS microphone (SPM1423), a small display, a single-cell 200 mAh lithium-polymer battery, and a 2.4 GHz Wi-Fi radio [**?**].

Two properties of this platform shape the entire data pipeline. The first is that the device is worn on the body, which rules out a high-bandwidth tethered connection and makes wireless streaming the practical way to obtain data from several body locations simultaneously. The second is the limited on-chip memory: the manufacturer datasheet gives 8 MB of flash, 2 MB of in-package PSRAM, and 520 KB of on-chip SRAM. This budget is comfortable for buffering and forwarding sensor samples during acquisition, but it is the binding constraint on the eventual on-device inference stage treated in Section 4.8. The dataset was therefore designed so that the same device that records the training data can, in principle, run the trained model: the inertial and acoustic sampling rates, the window length, and the feature representations described later in this chapter were all chosen with that target in mind rather than for offline convenience.

During data collection both the microphone and the IMU were used. The microphone is single-channel, so each device produced one monophonic audio stream alongside its six-axis inertial stream.



Figure 3.1: The M5StickC Plus2 wearable device. The ESP32-PICO-V3-02 system-on-chip, the six-axis IMU (MPU6886), and the microphone (SPM1423) are all integrated on the board. Image: M5Stack [**?**].

3.2 Assembly Scenario and Activity Taxonomy

Recordings were carried out in the Future Work Lab at Fraunhofer IPA in Stuttgart, a laboratory configured for research on human–machine interaction in manufacturing [CITATION]. The environment provides a workbench with parts, hand tools, and fasteners, so that a manual assembly task can be performed under controlled but realistic conditions.

Data were recorded from two participants over three sessions. In each session one participant wore the sensors and performed the task while the other operated a handheld mobile-phone camera, following the acting participant on foot during the segments that involved walking; the two participants exchanged roles for later sessions. Each session consisted of a repeated assembly sequence: three repetitions in two of the sessions and four repetitions, plus one sequence marked invalid, in the third. Session lengths, measured from the recorded streams, were approximately 6.7, 7.6, and 16.9 minutes, for a total of about 31 minutes of recording (Section 3.4).

Each repetition followed the same 32-step protocol, transcribed in the annotation records. The participant begins at the bench, walks to a supply table, and carries parts back; stacks three parts so that they align; seats them with a hammer; measures the assembly with a scale; passes four screws through the aligned parts by hand; drives the screws with a powered screwdriver; and places the finished item in a storage compartment beside the bench. The remainder of each repetition reverses the process, loosening and removing the screws and returning the parts to their boxes. Because the same operations recur several times per repetition, the screwdriving class dominates the dataset (Section 3.8).

Video of each session was annotated with a larger set of labels than the dataset finally uses. Across sessions the coded label set contained fifteen to eighteen activity labels, including narration (`speech`), the disassembly actions (`screw_loosening`,

remove_screws, store_part), a rare pick_container action, and in one session the postures sit and stand. The modelling taxonomy retains the ten classes listed in Table 3.1, which cover the assembly half of the task together with locomotion and rest. screw_loosening was merged into screw_tightening: the two actions use the same powered screwdriver and differ only in rotation direction, producing near-identical inertial and acoustic signatures that a two-second window cannot reliably separate. This merge accounts for the class's size: the 518 screw_tightening windows in Table 3.1 include both tightening and loosening instances. The remaining labels (speech, remove_screws, store_part, pick_container, and, in one session, sit/stand) were excluded rather than merged. The class descriptions in Table 3.1 are operational summaries taken from the 32-step protocol; the typical instance duration is the mean length of a coded activity segment, averaged over the three sessions.

Class	**Operational description (from the recording protocol)**	**Typ. dur.**	**Win.**
idle	Standing at the bench without performing a task.	4–12 s	45
walk	Walking between the workstation and the supply table.	≈5 s	73
carry_container	Carrying the parts container between the supply table and the workstation.	5–9 s	56
place_container	Setting the parts container down on the work surface.	3–5 s	35
assemble_parts	Stacking three parts on the workstation so that they align and fit together.	10–19 s	143
hammering	Striking the stacked parts with a hammer to seat them.	6–11 s	85
measuring	Measuring the length and width of the assembled part with a scale.	8–9 s	93
insert_screws	Passing four screws through the aligned parts by hand.	19–22 s	219
screw_tightening	Driving screws with a powered screwdriver; includes loosening actions merged into this class (Section 3.2).	22–31 s	518
compartment_interaction	Opening and closing the storage compartment beside the bench.	5–11 s	113
Total paired windows			**1380**

Table 3.1: Activity classes in the modelling dataset. Descriptions are taken from the 32-step assembly protocol. "Typ. dur." is the range of mean coded-segment durations across the three recording sessions; for screw_tightening it covers both the tightening and the merged loosening segments. "Win." is the number of two-second windows per class in the final dominant-wrist dataset (Section 3.8).

3.3 Data-Collection Firmware

The same custom firmware was installed on all of the devices, written in C++ and based on the Arduino framework. Its only task during acquisition is to sample the two sensors and forward every sample to a host computer with enough timing information for the streams to be reassembled and aligned afterwards. It performs no filtering, no feature extraction, and no on-device storage. This section explains the data acquisition architecture and the decisions behind it, rather than the source code.

3.3.1 Dual-Stream UDP Architecture

The inertial and acoustic data are transmitted as two separate streams over Wi-Fi using the User Datagram Protocol (UDP). One port receives inertial packets and another receives audio packets, so that a receiver does not need to inspect packet contents to separate the two modalities. UDP was preferred to a connection-oriented protocol since it introduces no retransmission or reordering delay, keeping the sender simple and the timing of transmitted packets close to the timing of the underlying samples; the trade-off is that lost packets are not retransmitted by the transport and must be detected and handled in post-processing [CITATION].

Each time a new IMU sample becomes available, an inertial packet is created that contains the six inertial values as text, a sequence number, and a device timestamp. Each audio packet has a sequence number, a sample count, a device timestamp, and a fixed-size block of pulse-code-modulated (PCM) samples. Since multiple devices send to the same host on the same two ports, the receiver logs the source address of each packet it receives and uses it to differentiate the streams by device. Figure 3.2 shows the end-to-end acquisition path.

3.3.2 Audio Double-Buffering and Packet Framing

Audio is captured continuously, so the firmware cannot stall the microphone driver while a packet is being transmitted. To avoid this, two 512-sample buffers are used alternately: while the driver fills one buffer, the previously filled buffer is transmitted, and once a buffer is full, the roles swap. This prevents new samples from overwriting data that has not yet been transmitted and keeps the capture cadence regular.

The buffer size also determines the size of each audio packet. A 512-sample block at a 16 kHz sampling rate corresponds to 32 ms of audio. Together with a header carrying the sequence number, sample count, and timestamp, this produces a packet of 1038 bytes, which is well below the 1500-byte Ethernet MTU (Maximum Transmission Unit), so every audio packet can be transmitted as a single, unfragmented UDP datagram. Keeping packets under the MTU matters because IP fragmentation of a UDP datagram means that losing any one fragment discards the whole datagram, which can increase the effective audio packet loss rate [CITATION].

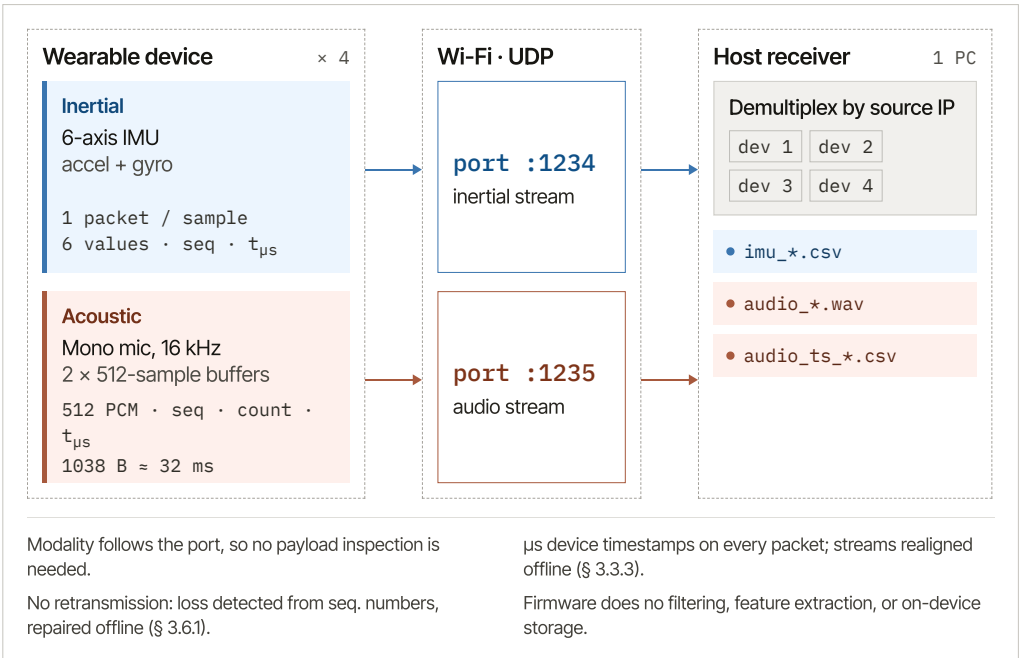


Figure 3.2: Architecture of the multimodal data-acquisition system. Each of the four wearable devices samples an IMU and a microphone independently, timestamps and frames the samples, and streams them as two separate UDP streams (inertial on one port, audio on another) to a single host, which separates the devices by packet source address and writes raw per-device files.

3.3.3 Microsecond Timestamping and Sequence Numbering

Each packet, inertial and acoustic, carries a timestamp taken from the device's microsecond counter at the moment the packet is assembled, together with a monotonically increasing sequence number. These two fields serve different purposes. The sequence number makes packet loss detectable: a gap between consecutive received sequence numbers means a lost packet, independent of any timing assumption. The timestamp makes the true timing of a recording session recoverable: it allows the elapsed time between the first and last samples to be determined, which is later used for clock-drift correction in Section 3.6. It also provides the temporal reference needed to align the inertial and acoustic streams from the same device.

The microsecond resolution is more than sufficient for this purpose: it is far finer than the interval between individual inertial samples and than the interval

between consecutive audio packets, so timestamp quantisation does not introduce a meaningful limitation to the alignment accuracy. The devices' timestamps cannot be compared directly with one another, however, since the individual devices do not share a common clock. The video reference described in Section 3.5 is therefore used to synchronise the various devices.

3.3.4 Multi-Device Capture and Body Placements

In each recording session, four sensor devices were worn: one on the dominant wrist, one on the non-dominant wrist, one at the waist, and one on the leg. All four devices simultaneously recorded inertial and acoustic data and streamed them to the host system. Recording at several body locations was a deliberate design choice: it made it possible to capture potentially useful information from different parts of the body and offered redundancy against a device failing during a recording session.

However, not all of the recorded data were used to train the model. Table 3.2 summarises the sensor data recorded and the data finally input to the modelling pipeline. The models presented in this thesis use only the dominant-wrist inertial data and the dominant-wrist microphone data.

During preprocessing, windows were first created for all four sensor locations and placed in their respective session directories. These windows used the same naming convention across all placements, based on the activity, window start time, and window index on the shared video-aligned timeline. This naming convention persisted when the windows were subsequently consolidated into one directory, so each window was retained as a single file. A byte-level comparison of the retained inertial and audio files indicated that they are those of the dominant-wrist device. Non-dominant-wrist, waist, and leg data are thus available in the original session data, but not in the modelling dataset.

The data from the leg-worn sensor package were excluded after inspection of the recorded signals. The leg inertial data contained relatively little activity-specific information for the bench-based assembly tasks, apart from activities such as walking. Likewise, the leg microphone primarily recorded ambient noise and captured the assembly activities poorly. This thesis also did not use the inertial data of the non-dominant wrist and waist as model inputs. Integrating information across more than one body location, at the feature level or at the decision level, is instead noted as a direction for future work, addressed in Chapter 7.

3.4 Recording Protocol

The same procedure was followed for each recording session. All four devices were worn by the acting participant and fastened with straps to keep the sensor axes fixed relative to the body segment to which they belonged. Before recording started, all devices were checked to ensure that they were connected and transmitting data correctly. The participant then repeated the set of predefined assembly steps at

Placement	Modalities recorded	IMU as model input	Audio as model input
Dominant wrist	inertial + audio	yes	yes
Non-dominant wrist	inertial + audio	no	no
Waist	inertial + audio	no	no
Leg	inertial + audio	no	no

Table 3.2: Body placements: data recorded during acquisition versus data used as model input in the final pipeline. Only the dominant-wrist inertial and acoustic streams reach the model.

the workbench while the second participant filmed the assembly sequence with a handheld phone. The host computer displayed the incoming packets from all four devices, so that a disconnection or a sustained loss of packets could be noticed; no session required a restart.

Table 3.3 provides an overview of the recorded sessions. The session durations were calculated from the recorded inertial streams, and the number of two-second windows was counted from the final dataset. The three sessions contributed 352, 371, and 657 windows. This class imbalance, as discussed in Section 3.8, is mostly due to the recording protocol: some actions, such as screwdriving, were performed several times within each repetition, whereas other actions, such as placing a container, occurred only once per repetition.

Part.	Session (project label)	Duration	Reps	Windows
P2	First session, all four placements	≈6.7 min	3	352
P1	Three repetitions	≈7.6 min	3	371
P2	Four repetitions, one invalid	≈16.9 min	4 (+1)	657
Total		**≈31.2 min**		**1380**

Table 3.3: Data-acquisition sessions. “Session” paraphrases the folder label used in the project records. “(+1)” denotes one further sequence marked invalid and set aside.

3.5 Ground-Truth Annotation from Synchronised Video

The sensor recordings provide motion and acoustic information but do not contain activity labels. The ground truth was therefore obtained from the recorded video,

which provides a visual reference for determining which activity was being performed at a given point in time. The annotated video intervals were then transferred to the sensor recordings. This process requires the video and sensor data to be placed on a common timeline and the activities to be represented as defined time intervals.

3.5.1 Determination of the Audio-Video Offset

The video and sensor recordings were started independently, meaning that their time origins were not identical. Before the video annotations could be mapped to the sensor data, a temporal offset between the two recordings had to be determined.

The offset was calculated by finding an event that is clearly identifiable in both the video and the recorded audio. The audio from the sensor side was analysed in Audacity [**?**] to find the corresponding transient, which was located within a millisecond. The same event, a spoken cue that the participant announced at the start and end of each assembly sequence, was then found in the video's audio track. The difference between the event times in the sensor recording and the video provides the required temporal offset.

For instance, the offset measured for one session is 3.857 seconds. This value is used for the synchronisation procedure to take out the same amount of data from the beginning of each sensor stream, so that the beginning of the sensor timeline becomes the zero point of the video. In each recording session, the offset was determined independently and applied to the synchronised streams used in this thesis. This step appears in the full preprocessing pipeline shown in Figure 3.3.

3.5.2 Interval Labels and Sample-Level Label Embedding

The activities were annotated directly from the video using BORIS [**?**], a tool for behavioural observation and event logging. Every annotation specifies a start time, an end time, and an activity label on the video timeline. The annotations were exported to a table with the columns `Label`, `Start`, and `End`.

These annotation intervals were then moved to the sensor data. For each inertial sample, its timestamp relative to the beginning of the recording was compared with the annotated intervals. If the timestamp was within an activity interval, the associated activity label was attributed to the sample. Samples that were not part of any of the annotated intervals were assigned a reserved `unlabeled` marker. This avoids assigning an activity label to transition periods or moments in which no specific task was being performed.

The assigned label was saved as a separate column along with the inertial measurements, and thus the inertial recording was transformed into a labelled time series. The audio stream was labelled using the same temporal reference. Since the audio and inertial data originate from the same device and were aligned to the video timeline, an audio segment corresponds to the activity represented by the inertial samples covering the same time interval. Consequently, transitions and untasked

periods remain explicitly represented as `unlabeled`. Their treatment during the subsequent windowing process is described in Section 3.7.

3.6 Signal Conditioning

Before the data were divided into windows, the recorded streams were processed to compensate for transmission-related artefacts and differences in the device clocks. Next, the inertial and acoustic data were brought to fixed sampling rates to ensure consistent processing across all the recordings.

The conditioning process was repeated for each of the devices and yielded continuous, uniformly sampled, and temporally aligned inertial and acoustic streams. The entire pre-processing chain is shown in Figure 3.3.

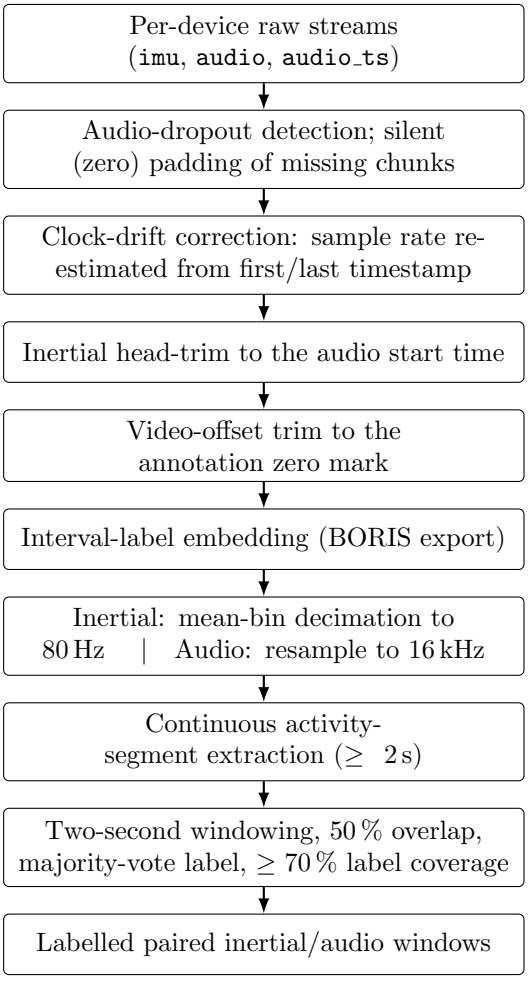


Figure 3.3: Signal-conditioning and windowing pipeline, applied per device. The inertial and acoustic branches are brought to fixed rates before both are cut on the same window boundaries.

3.6.1 Packet-Loss Detection and Audio-Dropout Reconstruction

Missing audio packets appear as gaps in the recorded waveform. Since each audio packet contains an arrival timestamp and a fixed number of samples, the expected time between consecutive packets can be determined. At a sampling rate of 16 kHz, for example, a packet containing 512 samples represents 32 ms of audio, so under normal conditions consecutive packets are expected to be separated by approximately 32 ms.

A larger time gap between two received packets indicates that one or more packets may have been lost. The number of missing packets is estimated from the observed time difference using

$$\mathrm{round}\left(\frac{\Delta t - T_{\mathrm{packet}}}{T_{\mathrm{packet}}}\right),$$

where Δt denotes the observed time difference between two consecutive received packets and T_{packet} denotes the expected packet interval. For the 16 kHz audio stream with 512 samples per packet, $T_{\mathrm{packet}} = 32\,\mathrm{ms}$. For example, if two received packets are separated by 96 ms, the equation gives

$$\mathrm{round}\left(\frac{96 - 32}{32}\right) = 2.$$

This means that two packets are assumed to be missing between the two received packets.

If a packet is detected as missing, a block of 512 zero-valued samples is inserted at the appropriate location in the waveform. This approach does not attempt to reconstruct the missing acoustic content. Instead, its purpose is to preserve the correct duration and temporal position of the surrounding signal so that subsequent time-frequency analysis is not affected by a shortened audio stream. The missing regions are therefore represented as silence. This is a conservative approach, as it introduces no artificial acoustic events, at the cost of a short period of artificially silent audio.

The fraction of reconstructed audio, computed from the raw per-device packet-arrival timestamps, is 0.15–0.40 % per session and about 0.33 % overall for the dominant wrist (Section 3.9); this is numerically identical to the packet-loss rate, since every missing packet is exactly 512 samples of silence.

The sequence numbers are used to detect the loss of packets in the inertial streams as well. In the dominant-wrist stream, the packet-loss rates were 0.056 %, 0.022 %, and 0.031 % for the three sessions, respectively, and roughly 0.035 % overall (Section 3.9). As described in Section 3.6.2, the short gaps in the inertial data were then filled by linear interpolation on the resampled time grid.

3.6.2 IMU Decimation to 80 Hz

The inertial stream is delivered at the device native rate, measured at 249.6–249.9 Hz across the three sessions from the devices' microsecond timestamps. This is consis-

tent with the recovered design notes but not with the firmware's own in-code comment, which labels the relevant configuration call "100 Hz"; the discrepancy has no effect on the modelling data, since decimation always targets a fixed 80 Hz regardless of the input rate. The stream is reduced to this uniform 80 Hz for modelling.

The reduction is performed by mean binning. The stream is divided into consecutive 12.5 ms bins, matching the desired sampling period at 80 Hz, and the inertial values in each bin are averaged to produce one output sample. If a bin contains no samples, for instance because of packet loss, its value is estimated by linear interpolation between the surrounding samples. The label column is decimated separately by taking the most frequent label in each bin. The resulting stream sits on an exact 80 Hz grid, which removes the small timing irregularities of the native stream.

Mean binning also provides a simple form of smoothing, although it is not used as a substitute for a specifically designed anti-aliasing filter. In particular, averaging within each bin attenuates signal components above the new Nyquist frequency of 40 Hz to some degree, but no filter with a defined passband or stopband was designed as part of this preprocessing step.

The rate of 80 Hz was selected based on the target platform and the anticipated nature of the activities to be captured. The sampling rate is intended to retain the frequency content relevant to the wrist movements involved in manual assembly, while substantially reducing the amount of inertial data and the resulting model input size [CITATION]. This reduces the inertial sampling rate by roughly a factor of three compared with the measured native rate.

3.6.3 Audio Resampling to a Strict 16 kHz Timeline

Following clock-drift correction, every audio stream is resampled from its effective sampling rate to a strict 16 kHz timeline using the band-limited resampler in librosa [**?**]. The resulting signal is stored as 16-bit PCM.

Using a fixed sampling rate ensures that the number of audio samples directly corresponds to a fixed duration. This is crucial for the windowing and feature-extraction stages that follow, because each example must span the same time interval and generate a consistent input representation. The sampling rate of 16 kHz is also sufficient to capture the frequency range considered relevant for the recorded assembly activities, which include impulsive and periodic sounds such as those generated when hammering and screwdriving. It is also consistent with the sampling rate used by the audio feature-extraction pipeline described in Chapter 4.

3.7 Window Generation and Labelling

After signal conditioning, the synchronised streams are first cut into continuous activity segments. Each segment is associated with one annotated activity interval and is only considered for further processing if it has a duration of at least two seconds. These segments are later split into equal-size windows for classification.

Each window has a duration of two seconds, with a 50 % overlap between consecutive windows, so a new window starts every one second. Overlapping windows are common in activity recognition because they increase the number of training examples and reduce the chance that a short activity is sliced at a window boundary [**?**]; the selected two-second window length and 50 % overlap are within the range commonly used in wearable HAR [**?**]. For the selected sampling rates, each two-second window contains 160 inertial samples at 80 Hz and 32 000 audio samples at 16 kHz. Both modalities are segmented using the same temporal boundaries on the shared timeline, so each inertial window has a corresponding audio window of the same span; the two together form one paired multimodal example.

The activity label of each window is decided using majority vote. The vote counts only samples that carry a valid activity label; samples tagged as `unlabeled` (Section 3.5.2) are not included. The activity that appears most often among the labelled samples is assigned to the window.

To prevent transition periods from becoming part of the training data, a minimum label-coverage threshold of 70 % is applied. A window is kept only if at least 70 % of its samples have a valid activity label. A window with a high proportion of `unlabeled` samples is not assigned to an activity class but is discarded instead, usually at transitions between activities. This method keeps windows with many transitions out of the training data while preserving windows that may have some activity-boundary overlap. Figure 3.4 shows the full window generation and labelling process.

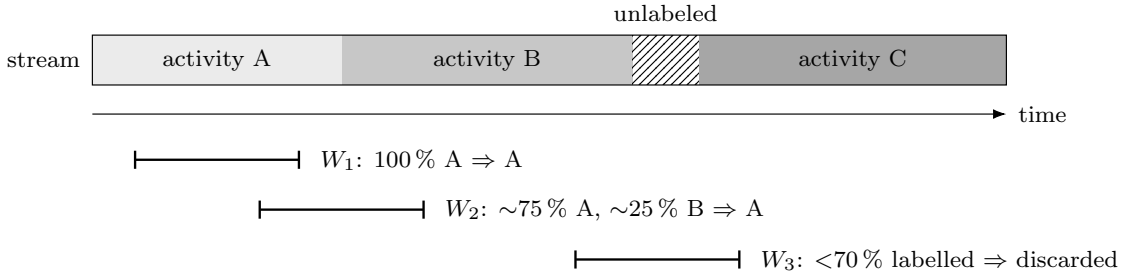


Figure 3.4: Window generation and majority-vote labelling. Two-second windows advance in one-second steps (W_1, W_2, W_3 shown stacked for clarity). A window takes the majority activity among its labelled samples; a window whose labelled coverage falls below 70 %, such as one spanning the transition region, is discarded.

Windowing was carried out for all four body placements, producing one set of labelled inertial and audio windows per placement, organised by class. The per-placement sets that are not used for modelling contain fifteen activity labels each.

3.8 Dataset Statistics, Class Distribution, and Imbalance

The dataset used for modelling is the dominant-wrist subset. It has 1380 paired inertial and audio windows for the ten activity classes outlined in Table 3.1. The data come from two participants and three recording sessions and represent roughly 23 minutes of windowed activity out of the 31 minutes of recorded data; the remaining time is not part of the modelling dataset because of excluded classes, the invalid sequence, and windows removed around activity transitions.

Each inertial window contains a 160×6 array of accelerometer and gyroscope measurements sampled at 80 Hz. Each corresponding audio window contains 32 000 mono samples at 16 kHz. Table 3.4 summarises the composition of the dataset, and Figure 3.5 shows the distribution of windows across the activity classes.

Property	**Value**
Paired inertial/audio windows	1380
Activity classes	10
Participants	2
Recording sessions	3
Windowed activity	≈23 min
Window length	2 s
Window stride	1 s (50 % overlap)
Minimum label coverage	70 %
Inertial window	160×6 at 80 Hz
Audio window	32 000 samples, mono, 16 kHz
Inertial channels	acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z

Table 3.4: Composition of the dominant-wrist modelling dataset.

There is a significant imbalance in the class distribution. The smallest class, `place_container`, contains only 35 windows, roughly one fifteenth of the largest. This imbalance is primarily due to the design of the task: actions such as screwdriving were repeated numerous times during each session, while other activities such as moving a container or walking were repeated only a few times. Consequently, the imbalance persists in the dataset and is not corrected during preprocessing. How it is handled in training and evaluation is detailed in Chapter 4, and its impact on the findings in Chapter 6.

The number of windows discarded during windowing because they failed the 70 % label-coverage requirement was not recorded.

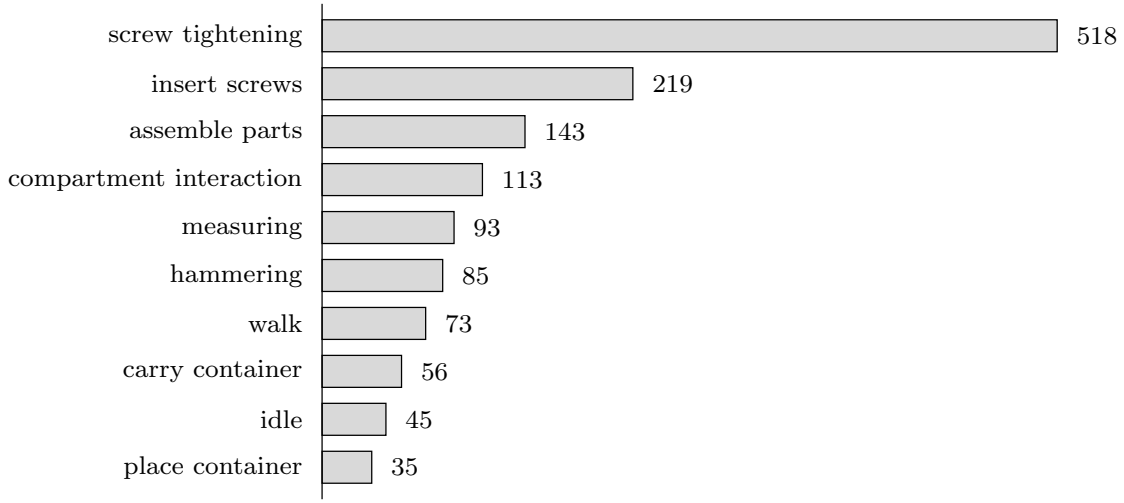


Figure 3.5: Number of two-second windows per activity class in the dominant-wrist dataset ($n = 1380$). The distribution is dominated by the screwdriving classes; `screw_tightening` alone accounts for about 38 % of the windows and the two screwdriving classes together for more than half.

3.9 Data-Quality Assessment and Known Artefacts

There are a number of characteristics of the data that are important to consider. These characteristics do not prevent the dataset from being used for the experiments in this thesis, but they define the limits of what can reasonably be concluded from those experiments. Table 3.5 summarises the primary data-quality indicators that could be established.

Indicator	**Value**	**Notes**
Inertial packet loss (dominant wrist)	0.022–0.056 % per session	From sequence-number gaps; ≈0.035 % overall. Below the 1 % design target.
Audio packet loss (dominant wrist)	0.15–0.40 % per session; ≈0.33 % overall	From `audio_ts` packet-arrival timestamp gaps, computed from the raw per-device logs. Reconstructed audio (inserted silence) is the same fraction, since every missing packet is exactly 512 samples.
Synchronisation basis	one manual event per session	Residual reading error and slow clock drift over a session are not characterised.
Camera	handheld, moving	Framing varied, but no hand action was occluded during annotation.
Discarded low-coverage windows	not recorded	See Section 3.8.

Table 3.5: Data-quality indicators for the dataset.

Packet loss and audio reconstruction. Wireless UDP transmission loses packets. Inertial loss on the dominant-wrist stream is measured at about 0.035 % overall

(Section 3.6.1). Audio loss, computed from the raw per-device packet-arrival timestamps, is higher: 0.15–0.40 % per session and about 0.33 % overall for the dominant wrist, roughly an order of magnitude above the inertial loss rate. Because every missing audio packet is reconstructed as exactly 512 samples of silence (Section 3.6.1), the reconstructed-audio fraction is numerically the same figure.

Synchronisation uncertainty. The alignment of the sensor recordings with the activity annotations is based on a manually identified synchronisation event for each session, followed by a fixed offset applied to the entire session (as described in Section 3.5.1). There is no quantification of the residual error caused by manual event locating, and no characterisation of the gradual clock drift between the recordings. Majority voting applied over two-second windows and the 70 % threshold for label coverage minimise the risk of a small window alignment error affecting the label assigned to a particular window. However, this kind of error can still change the effective boundaries of an activity.

Scope of the dataset. The dataset was collected from two participants performing a scripted assembly task in a single laboratory environment. The final modelling data represent approximately 23 minutes of windowed activity, with one class accounting for more than one third of all windows. The dataset is suitable for the comparative analysis of multimodal fusion methods and on-device feasibility undertaken in this thesis. Its size and limited variation do not make it suitable for assessing how the resulting models would perform for different participants, assembly tasks, or environments. These limitations are addressed in more detail in Chapter 6, which also discusses their effect on the experimental results.

3.10 Chapter Summary

This chapter described the construction of the multimodal dataset. Inertial and acoustic data were streamed over Wi-Fi from four M5StickC Plus2 devices worn at the wrists, waist, and leg, using a UDP-based firmware that timestamps and sequence-numbers every packet so that the streams can be realigned after transmission. Two participants performed a scripted manual assembly task across three sessions in the Fraunhofer IPA Future Work Lab, alternating between performing the task and filming it on a handheld phone. Activity intervals were coded from the video in BORIS and transferred onto the sensor timeline after the sensor-to-video offset was fixed from a shared event located in Audacity. Each device stream was then corrected for audio dropouts and clock drift, decimated to 80 Hz for the inertial channels and resampled to 16 kHz for audio, and segmented into overlapping two-second windows labelled by majority vote subject to a 70 % coverage threshold. The modelling dataset used in the remainder of this thesis is the dominant-wrist subset: 1380 paired inertial and audio windows over ten activity classes, single-channel audio, with a strong class imbalance dominated by screwdriving actions and with the data-quality limitations set out in Section 3.9. This dataset is the common input to all of the models and experiments described in the following chapters.

4 Model Design, Training and Deployment

This chapter describes the models used for activity recognition and the procedure used to train and evaluate them. It begins with an overview of the experimental design and its relation to the research questions. It then covers the input representations derived from the inertial and acoustic windows of Chapter 3, the single-modality models, the three fusion strategies, and a classical baseline using hand-crafted features. It closes with the training procedure, the deployment of selected models on the wearable device and the evaluation protocol that all models share. The results obtained with these models are presented in Chapter 5.

4.1 Overview of the Experimental Design

The experiments compare models that differ in one respect at a time. All models are trained and evaluated on the same windows and the same train/validation split, and they use the same input normalisation and the same training settings. Differences in accuracy can therefore be attributed to the model design and not to differences in the data or in the training procedure.

The models fall into four groups, each linked to one or more of the research questions formulated in Chapter 1:

- **Single-modality models** use either inertial or acoustic data alone. They form the reference point against which the benefit of adding audio is measured (RQ1, RQ3).
- **Fusion models** combine both modalities at different stages of processing. They are combined at the input level (early fusion), at the level of learned representations (embedding-level fusion), or at the level of class predictions (decision-level fusion). Their comparison addresses RQ2.
- **A classical baseline** uses conventional machine learning without deep neural networks. Simple statistical features are computed by hand from each window and classified with a random forest. Comparing the neural networks with this baseline shows how much is gained by learning the features from the data instead of defining them in advance (RQ3).
- **Reduced model variants** with fewer parameters were designed with the memory and computational limits of the target device in mind. They connect

the offline comparison to the deployment on the device described in Section 4.8 (RQ4).

Group	**Model**	**Inputs**	**Fusion stage**
Single modality	IMU CNN	inertial	–
Single modality	Audio CNN (GAP, GRU and hybrid pooling)	acoustic	–
Early fusion	Channel concatenation (standard and small)	inertial + acoustic	input
Early fusion	Unified spectrogram	inertial + acoustic	input
Embedding-level fusion	Frozen backbones with a trained fusion head (standard and small)	inertial + acoustic	representation
Decision-level fusion	Weighted average with global or per-class weights (standard and small)	inertial + acoustic	prediction
Classical baseline	Random forest on hand-crafted features	inertial + acoustic	feature

Table 4.1: Overview of the evaluated models.

The choice of model types follows from two requirements of this thesis. The models must recognise activities from short windows of sensor data, and the best of them must later run on a microcontroller with very limited memory and computing power. Convolutional neural networks (CNNs) meet both requirements. They learn characteristic local patterns directly from the signal, such as the shape of a hammer strike in the acceleration or the frequency content of a screwdriver in the spectrogram, and they need comparatively few parameters because the same small filters are applied across the whole window. For these reasons, CNNs are among the most widely used architectures in wearable activity recognition and in embedded audio classification [CITATION]. Recurrent layers, which process a sequence step by step, can capture the temporal order of a signal more explicitly but are more expensive to execute [CITATION]. They are therefore included only as an optional pooling stage of the audio model, so that their benefit can be measured against their cost. The three fusion strategies cover the three stages at which modalities are commonly combined in multimodal learning [CITATION]. The random forest, finally, is a standard reference method in activity recognition that performs well on small datasets and requires little tuning [CITATION].

All experiments use the dominant-wrist dataset described in Chapter 3, with 2565 paired windows across nine activity classes. The only exception is the standalone audio model, which additionally uses audio windows from the non-dominant wrist and the waist. The reason for this is given in Section 4.3.2.

4.2 Input Representations

4.2.1 Inertial Windows

Each inertial window holds 160 samples of the six channels (three-axis acceleration and three-axis angular velocity) recorded at 80 Hz. Before entering a model, the window is resampled to 200 time steps by linear interpolation along the time axis, so that all inertial models take inputs of size 200×6. The target length of 200 was chosen to match the time resolution of the audio spectrogram, which, as described in Section 4.2.2, has 198 frames per window. This allows the early fusion model presented in Section 4.4.1 to place both modalities on one time axis without coarsening the audio representation. Using the same length in all inertial models also keeps their inputs identical across the experiments. Missing or non-finite values, which can remain at the very edges of a recording after decimation, are replaced by zero.

Accelerometer and gyroscope channels have different physical units and value ranges. Each channel is therefore standardised to zero mean and unit variance. The mean and standard deviation of each channel are computed on the training split only and then applied unchanged to the validation data, so that no statistics of the validation set enter the training process. As described in Section 4.8, the same statistics are later embedded in the device firmware and applied to the live sensor data.

4.2.2 Audio Log-Mel Spectrograms

Raw audio windows of 32 000 samples are too long to be used directly as input to a small model. Each window is therefore converted into a log-mel spectrogram, a time-frequency representation that is widely used for sound classification [CITATION]. The conversion consists of three steps. First, a short-time Fourier transform (STFT) splits the signal into overlapping frames of 25 ms (400 samples), shifted by 10 ms (160 samples), and computes the frequency spectrum of each frame with an FFT length of 512. Second, the magnitude spectrum of each frame is projected onto 40 frequency bands between 20 Hz and 8 kHz. The bands are spaced according to the mel scale, which resolves low frequencies more finely than high ones, similar to human hearing [CITATION]. Third, the natural logarithm of the band energies is taken. A small constant of 10^{-6} is added beforehand so that silent bands do not produce the logarithm of zero.

A two-second window produces a spectrogram of 198 frames $\times$ 40 mel bands. This reduces the input size by roughly a factor of four compared with the raw waveform, while keeping the spectral and temporal structure that distinguishes, for example, the continuous whine of a powered screwdriver from the short, broadband impacts of hammering. As with the inertial data, the spectrogram is standardised per mel band using the mean and standard deviation of the training split. Figure 4.1 summarises the feature extraction, and Table 4.2 lists the resulting input representations.

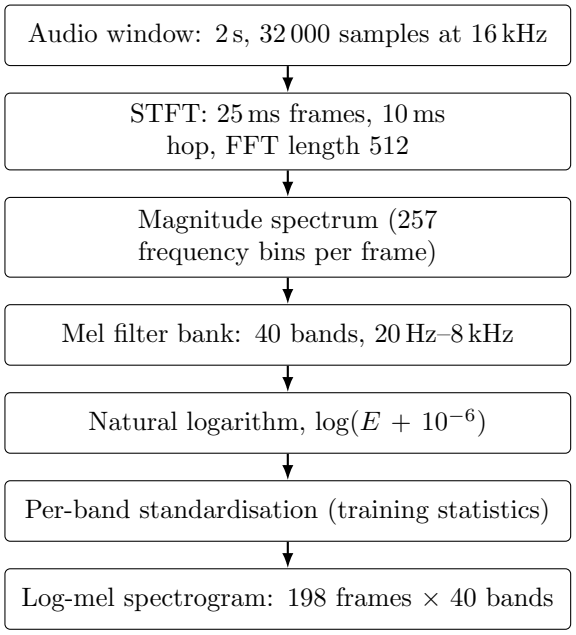


Figure 4.1: Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models.

Modality	**Raw window**	**Model input**	**Normalisation**
Inertial	160×6 at 80 Hz	200×6 (linear resampling)	per channel, training statistics
Audio	32 000 samples at 16 kHz	198×40 log-mel	per mel band, training statistics

Table 4.2: Input representations of the two modalities.

4.3 Single-Modality Models

4.3.1 Inertial Model

The inertial model is a one-dimensional convolutional neural network (1D CNN) that convolves along the time axis of the window. 1D CNNs are a common choice for wearable activity recognition because they learn local motion patterns directly from the raw signal and remain small enough for embedded use [CITATION].

The network has two convolutional blocks. The first block applies 64 filters with a kernel size of 5, followed by batch normalisation and max pooling with a factor of 2. The second block applies 128 filters with a kernel size of 3, followed by batch normalisation. Global average pooling then reduces the temporal dimension to a single feature vector. This vector passes through a dense layer with 64 units and through dropout with a rate of 0.3, before a softmax output layer with one unit per class. The output of the 64-unit layer also serves as the embedding for the embedding-level fusion described in Section 4.4.2.

4.3.2 Audio Model and Temporal Pooling Variants

The audio model treats the log-mel spectrogram as a single-channel image and processes it with a two-dimensional CNN [CITATION]. It has two convolutional blocks with 32 and 64 filters of size 3×3, each followed by batch normalisation and 2×2 max pooling. The resulting feature map has a size of $49 \times 10 \times 64$ (time $\times$ frequency $\times$ channels).

How this feature map is reduced to a single vector determines which properties of the sound the classifier can use. Three variants were therefore evaluated, which are shown in Figure 4.2.

- **Global average pooling (GAP).** The feature map is averaged over both time and frequency [CITATION]. This is the standard approach and gives a compact model. However, averaging over time removes the information about how the sound develops within the window. Two activities with a similar average spectrum but a different temporal pattern, such as the regular strikes of a hammer and the steady sound of a screwdriver, can then become hard to separate.

- **Recurrent pooling (GRU).** The feature map is reshaped into a sequence of 49 time steps with 640 features each and passed through a bidirectional gated recurrent unit (GRU) with 32 units per direction [CITATION]. The GRU summarises the sequence while keeping its temporal order, so the model can use rhythm and envelope information.

- **Hybrid pooling.** The GAP vector and the GRU output are computed in parallel and concatenated. The GAP vector describes the average timbre of the

window, and the GRU output describes its temporal structure. The classifier can then rely on whichever information is useful for a given class.

In all three variants, the pooled vector passes through a dense layer with 64 units and dropout with a rate of 0.4, followed by the softmax output.

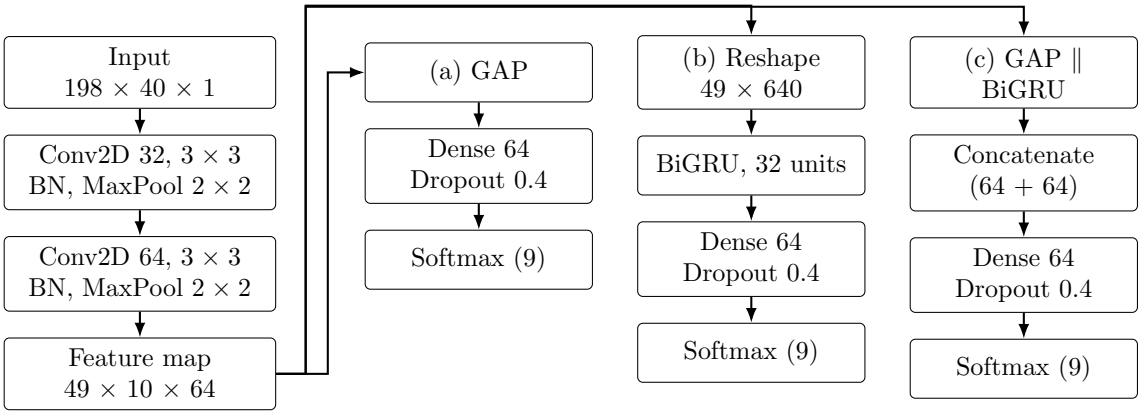


Figure 4.2: Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries.

Because audio content depends less on where the sensor is worn than inertial data does, the standalone audio model is trained on audio windows from the dominant wrist, the non-dominant wrist and the waist. This roughly triples the amount of acoustic training data. The pooled audio is used only for the standalone audio model. All fusion models use dominant-wrist audio exclusively, so that every audio window is paired with the inertial window from the same device and time interval.

The pooling variants are compared on the standalone audio model. Within the fusion models, the acoustic component uses the GAP variant. With 23 945 parameters it is about six times smaller than the recurrent variants, and it avoids recurrent layers, whose sequential computation is costly on the target microcontroller. This keeps all fusion models within reach of the deployment described in Section 4.8. The performance of the three variants is reported in Chapter 5. Table 4.3 summarises both single-modality architectures.

4.4 Fusion Architectures

The three fusion strategies differ in the point along the processing chain at which the two modalities are combined [CITATION]. Figure 4.3 gives an overview.

Layer	IMU CNN	Audio CNN
Input	200×6	$198 \times 40 \times 1$
Block 1	Conv1D 64, kernel 5, BN, MaxPool 2	Conv2D 32, 3×3, BN, MaxPool 2×2
Block 2	Conv1D 128, kernel 3, BN	Conv2D 64, 3×3, BN, MaxPool 2×2
Pooling	Global average	GAP / BiGRU(32) / GAP + BiGRU
Head	Dense 64, Dropout 0.3	Dense 64, Dropout 0.4
Output	Softmax (9 classes)	Softmax (9 classes)
Parameters	36 297	23 945 (GAP) / 153 353 (GRU) / 157 449 (hybrid)

Table 4.3: Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations.

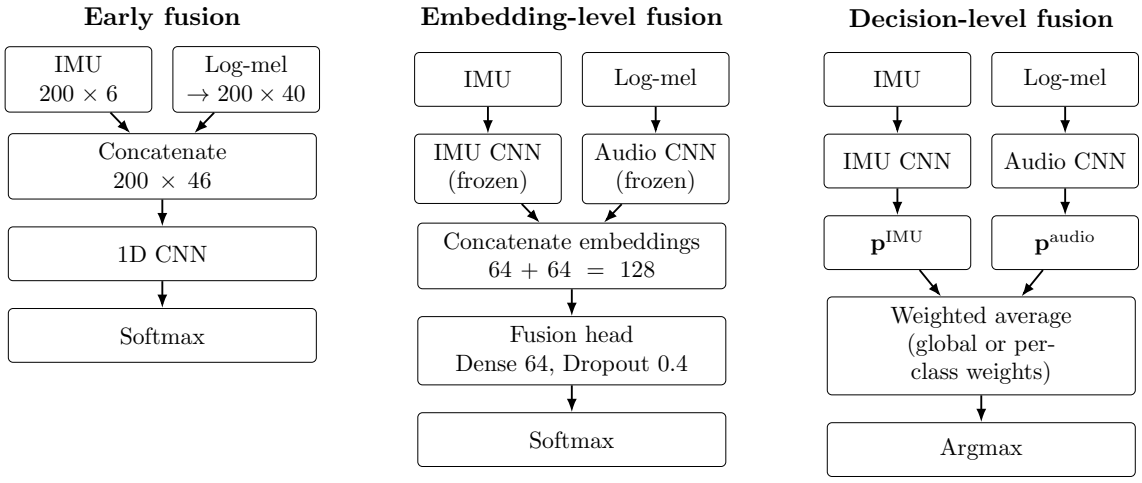


Figure 4.3: The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models.

4.4.1 Early Fusion

Early fusion combines the two modalities before any learning takes place, so that a single network learns from both signals jointly. Two forms were implemented.

Channel concatenation. The log-mel spectrogram (198×40) is resampled along its time axis to 200 frames, matching the length of the inertial window. The two inputs are then concatenated along the feature axis, giving a 200×46 input with 6 inertial and 40 acoustic channels. Each of the 46 channels is standardised with its own training statistics. This step is particularly important here, because the channels of the fused input have very different value ranges. In the recorded data, the acceleration lies roughly between -1.5 and $+1.5\,\mathrm{g}$ and the angular velocity between -200 and $+200\,^\circ/\mathrm{s}$, while the log-mel values lie roughly between -14 and $+3$. A convolutional filter sums weighted values from all channels. Without standardisation, the gyroscope channels would therefore dominate these sums simply because of their larger numbers, and the network would first have to learn to compensate for the different units before it could learn from the content of the signals. After standardisation, every channel has zero mean and unit variance and starts with the same influence on the result. The fused input is then processed by a 1D CNN with the same structure as the inertial model described in Section 4.3.1.

Unified spectrogram. Here the inertial data are also converted into spectrograms, so that both modalities share one image-like representation. A short-time Fourier transform with a frame length of 20 samples, a hop of 2 samples and an FFT length of 32 is applied to each of the six inertial axes. The log magnitudes are resized to the 198×40 grid of the audio spectrogram by bilinear interpolation. The six inertial spectrograms and the audio spectrogram are stacked as the seven channels of a single input and processed by the 2D audio CNN with global average pooling.

4.4.2 Embedding-Level Fusion

Embedding-level fusion first trains the inertial and the acoustic model separately, as described in Section 4.3. Their weights are then frozen, and for each window the outputs of the 64-unit dense layer are taken as embeddings. The two 64-dimensional embeddings are concatenated into a 128-dimensional vector, which is the input to a small fusion head. The head consists of a dense layer with 64 units, dropout with a rate of 0.4 and a softmax output, with 8841 parameters in total. Only the fusion head is trained in this second stage. It can learn how the two modalities relate to each other without changing the feature extraction that each backbone has already learned.

4.4.3 Decision-Level Fusion

Decision-level fusion combines the predictions of the two independently trained models [CITATION]. Both models end in a softmax layer, which outputs one value be-

tween 0 and 1 for each of the nine classes, with all nine values summing to 1. These values can be read as the confidence of the model that the window belongs to the respective class. For a window x, let $p_c^{\mathrm{IMU}}(x)$ denote the probability that the inertial model assigns to class c, and $p_c^{\mathrm{audio}}(x)$ the probability that the acoustic model assigns to the same class. The fused score of class c is the weighted average of the two,

$$p_c(x) = w \cdot p_c^{\mathrm{IMU}}(x) + (1 - w) \cdot p_c^{\mathrm{audio}}(x), \qquad w \in [0, 1]. \tag{4.1}$$

The weight w determines how much the inertial model is trusted relative to the acoustic model. With $w = 1$, only the inertial model is used, with $w = 0$, only the acoustic model is used, and with $w = 0.5$, both count equally. The fused score is computed for every class, and the class with the highest score is the prediction. The weight is selected by trying the 21 values $0, 0.05, \ldots, 1$ and keeping the one that gives the highest accuracy on the validation data. Compared with a trained fusion head, this method has only a single free parameter and is therefore less prone to overfitting on a small dataset.

A single weight, however, assumes that the relative reliability of the two modalities is the same for every class. This assumption does not hold for the present activity set. For some activities the motion pattern is distinctive while the sound is not. Standing idle and measuring, for example, produce hardly any sound. For others the opposite is true, and the powered screwdriver is the clearest example of an activity with a distinctive acoustic signature. The method was therefore extended to one weight per class. With $C = 9$ classes, this gives nine weights $w_1, \ldots, w_9$, and the fused score of class c becomes

$$p_c(x) = w_c \cdot p_c^{\mathrm{IMU}}(x) + (1 - w_c) \cdot p_c^{\mathrm{audio}}(x). \tag{4.2}$$

The only difference to Equation 4.1 is that the weight now carries the index c. When the fused model computes the score of a particular class, it uses the weight of that class, and it can therefore rely on whichever sensor is more informative for that class. A value of w_c close to 1 means that the score of class c is determined mainly by the inertial model, and a value close to 0 means that it is determined mainly by the acoustic model.

A short example illustrates the effect. Consider a window of the class `measuring`, for which the two models produce the probabilities shown in Table 4.4. For readability, only three classes are shown. The inertial model recognises the activity, whereas the acoustic model hears almost silence and leans towards `idle`. With a global weight of $w = 0.5$, the correct class wins only by a small margin. If the weight of `measuring` is raised to $w_{\mathrm{measuring}} = 0.9$, its score is taken almost entirely from the more reliable inertial model, and the margin increases clearly. The score of `idle` does not change, because its own weight has not changed.

It remains to find suitable values for the nine weights. For the single weight of Equation 4.1, all 21 candidate values could simply be tried. For nine weights, trying every combination would require 21^9, or about 800 billion, evaluations, which is not practical. The weights are therefore adjusted one at a time, while all others are

	`measuring`	`idle`	`hammering`
$\mathbf{p}^{\text{IMU}}$	0.70	0.25	0.05
$\mathbf{p}^{\text{audio}}$	0.20	0.60	0.20
Global weight, $w = 0.5$	**0.45**	0.425	0.125
Per-class weight, $w_{\text{measuring}} = 0.9$, others 0.5	**0.65**	0.425	0.125

Table 4.4: Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments.

kept fixed. This simple strategy is known as coordinate ascent [CITATION]. The name refers to the fact that only one coordinate of the weight vector, that is, one class weight, is changed in each step, and that a change is kept only if it raises the accuracy. The search proceeds as follows.

1. All nine weights start at the best single weight found for Equation 4.1. The search therefore begins with the result of global fusion and can only improve on it.

2. The weight of the first class is set in turn to each of the 21 values $0, 0.05, \ldots, 1$, while the other eight weights stay unchanged. For each value, all validation windows are classified with Equation 4.2, and the accuracy is computed. The value with the highest accuracy is kept, provided that it is better than the accuracy before this step.

3. Step 2 is repeated for the second class, then for the third class, and so on, until each of the nine weights has been adjusted once.

4. Since a change in one weight can make a different value of an earlier weight slightly better, steps 2 and 3 are carried out a second time.

In total, the search therefore evaluates $2 \times 9 \times 21 = 378$ weight settings, which takes less than a second. A second round is sufficient because the weights hardly influence each other. Changing the weight of one class only affects windows for which this class and another class received almost the same score. For most windows, one class is clearly ahead, and changing a single weight does not alter the prediction. The resulting weights show, for each class, how much the fused model relies on the inertial and how much on the acoustic model. They are therefore also used to interpret the results in Chapter 5. Figure 4.4 summarises the procedure.

4.5 Classical Baseline

To place the neural networks in context, a classical machine-learning approach was also evaluated. In contrast to the deep learning models, which learn their features

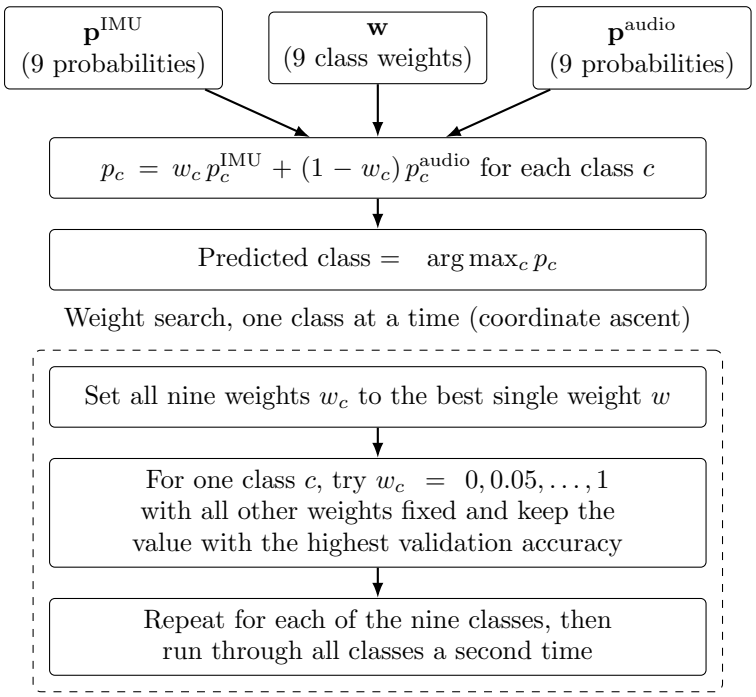


Figure 4.4: Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom).

from the data during training, this approach relies on hand-crafted features, that is, statistics whose definition is fixed in advance. For each of the six inertial channels, the mean, standard deviation, minimum, maximum, root mean square and zero-crossing rate are computed, which gives 36 inertial features. For each of the 40 mel bands of the log-mel spectrogram, the mean and standard deviation over time are computed, which gives 80 acoustic features. As listed in Table 4.5, the two feature sets are concatenated into a 116-dimensional vector, so the baseline also combines both modalities at the feature level.

A random forest with 300 trees is trained on these vectors, with balanced class weights to account for the class imbalance [CITATION]. Random forests need no feature scaling and give robust results on small tabular datasets, which makes them a common reference point in activity recognition [CITATION].

4.6 Deployment-Oriented Model Variants

The models described so far were designed for offline evaluation. On the target microcontroller, however, inference time and memory usage are strict limits. The TensorFlow Lite Micro build for this platform executes convolutions with unaccelerated reference kernels, so their cost grows directly with the number of filters and with the size of the feature maps. This is explained in more detail in Section 4.8.1.

Modality	Features	Count
Inertial	mean, standard deviation, minimum, maximum, RMS and zero-crossing rate for each of the 6 channels	36
Audio	mean and standard deviation over time for each of the 40 mel bands	80
Total		**116**

Table 4.5: Hand-crafted features of the classical baseline.

Reduced variants were therefore created that keep the overall architecture but use fewer filters.

- **Small early fusion model.** The structure of the channel-concatenation model with 32 and 64 instead of 64 and 128 filters. This reduces the number of multiply-accumulate operations, and with it the inference time, by roughly a factor of four.
- **Small inertial backbone.** A 1D CNN with 32 and 64 filters.
- **Small audio backbone.** A 2D CNN with 16 and 32 filters. Its convolutions use strides of 4 and 2 instead of separate pooling layers, so the downsampling happens within the convolution itself. As a result, a feature map at the full 198×40 resolution never has to be held in memory, which would otherwise take up a large part of the device's working memory.

The small backbones are used for the small embedding-level and decision-level fusion models. To reduce overfitting, three regularisation measures were added to them [CITATION]. L2 weight decay with a factor of 10^{-4} is applied to all learned layers, the dropout rates are raised to 0.4 and 0.5, and the loss function uses label smoothing of 0.1.

In addition, the training data of the small backbones are augmented. One augmented copy is generated for each training window, so that the training set doubles in size. The validation data are never augmented.

- **Inertial windows** receive a small random three-dimensional rotation of up to $\pm 15°$, applied jointly to the accelerometer and gyroscope axes to mimic small changes in how the device sits on the wrist, together with per-channel scaling (factor 0.9–1.1), additive noise (3 % of the channel's standard deviation) and a mild time warp (speed factor 0.9–1.1) [CITATION].
- **Spectrograms** are shifted in time by up to ± 10 frames, with the frames that leave one end of the window re-entering at the other. In addition, random parts of the spectrogram are masked, following the SpecAugment method originally proposed for speech recognition [CITATION]. Two blocks of up to 20 consecutive frames and two blocks of up to 6 adjacent mel bands are overwritten with

the mean value of the spectrogram. Because the hidden parts change from copy to copy, the model cannot depend on a single short moment or a single frequency range and has to use the information of the whole spectrogram.

Table 4.6 compares the standard and the small variants.

Model	**Standard**	**Small**
Early fusion (concatenation)	Conv1D 64 / 128 49 097 parameters	Conv1D 32 / 64 18 729 parameters
IMU backbone	Conv1D 64 / 128 36 297 parameters	Conv1D 32 / 64, L2, dropout 0.4 12 329 parameters
Audio backbone	Conv2D 32 / 64 with max pooling 23 945 parameters	Conv2D 16 / 32 with strides 4 / 2, L2, dropout 0.5 7 689 parameters
Label smoothing	–	0.1 (backbones)
Augmentation	–	one augmented copy per training window (backbones)

Table 4.6: Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer.

4.7 Training Procedure

4.7.1 Software and Hardware Environment

All models were implemented in Python 3.10 with TensorFlow 2.10 and Keras [CITATION]. Training ran on an NVIDIA GeForce RTX 4070 Laptop GPU through the DirectML backend for Windows. This backend supports TensorFlow only up to version 2.10, which fixed the framework version for the whole project. Two parts of the pipeline could not run on the GPU with this backend and had to be adapted. The short-time Fourier transform used for the spectrograms was moved to the CPU, and the fused recurrent kernel of the GRU layer was replaced by an unrolled implementation. Both changes affect only where and how the computation is executed, not its result. Data preparation used NumPy, pandas and librosa [CITATION], and the classical baseline used scikit-learn [CITATION].

4.7.2 Training Settings

All neural networks were trained with the settings listed in Table 4.7. The class imbalance described in Section 3.8 is handled through class weights in the loss function. The weight of each class is inversely proportional to its frequency in the training

split, so that errors on rare classes count more than errors on frequent ones [CITATION]. The training data themselves are left unchanged, without oversampling or undersampling.

Setting	**Value**
Optimiser	Adam [CITATION]
Learning rate	10^{-3}, constant
Batch size	32
Loss	Categorical cross-entropy (label smoothing 0.1 for the small backbones)
Class weighting	Inverse class frequency ("balanced")
Epochs	150 for backbones and early-fusion models, at most 80 for the fusion head
Model selection	Weights of the epoch with the highest validation accuracy
Fusion head	Early stopping (patience 15), learning rate halved on plateau (patience 6, minimum 10^{-5})

Table 4.7: Training settings shared by all neural network models.

The backbones and the early-fusion models are trained for a fixed 150 epochs at a constant learning rate. On a small validation set, the validation accuracy can fluctuate noticeably from one epoch to the next, so the weights at the end of training are not necessarily the best ones. After training, each model is therefore set back to the weights of the epoch with the highest validation accuracy.

In the Keras version used here, the standard early-stopping callback restores the best weights only when it actually stops training early. If training runs the full number of epochs, the weights of the last epoch are kept without any warning. A small custom callback was therefore written that stores the best weights during training and restores them at the end. The effect of this choice is examined in Chapter 5. Because the best epoch is selected on the validation data, the procedure introduces a slight optimistic bias into the reported results, a point taken up in the discussion of limitations in Chapter 6.

4.8 Deployment on the Wearable Device

The models described so far were trained and evaluated on a desktop computer. To answer RQ4, selected models were also brought onto the M5StickC Plus2, the same device that recorded the training data. Two conditions must hold for this to succeed. The model has to fit into the memory of the microcontroller and run fast enough for continuous use. In addition, the device has to transform the live sensor signals in exactly the same way as the training pipeline did, because even a correct model produces wrong predictions if its input differs from what it saw during

training. This section describes how both conditions were addressed. The measured memory use, inference time and on-device accuracy are reported in Chapter 5.

4.8.1 Target Platform and Software Stack

As described in Section 3.1, the device is built around the ESP32-PICO-V3-02, a dual-core processor clocked at up to 240 MHz with 520 KB of on-chip SRAM. Only part of this memory is available to the application, because the Arduino framework and the device libraries reserve their own share. The firmware was developed with PlatformIO and the Arduino framework. Neural network inference uses TensorFlow Lite for Microcontrollers [CITATION], a version of TensorFlow Lite designed for devices without an operating system and without dynamic memory allocation. The FFT for the audio features is computed with the arduinoFFT library [CITATION].

The TensorFlow Lite Micro port available for this platform does not include the optimised neural network kernels that the chip manufacturer provides for newer processors. All layers are therefore executed with the generic reference implementations. This is the reason why the inference time grows directly with the number of filters and the size of the feature maps, and why the reduced model variants described in Section 4.6 were developed.

Three firmware variants were implemented, one for each fusion strategy. They share the same capture and preprocessing code but differ in the number of networks they run, as summarised in Table 4.8. Each variant is a separate PlatformIO build environment, so that the data-collection firmware of Chapter 3 and the three inference variants can be flashed independently of each other.

Variant	**Networks on the device**	**Network inputs**	**Tensor arena**
Early fusion (small)	one fused network	200×46	40 KB
Embedding-level fusion (small)	IMU backbone, audio backbone and fusion head	200×6, 198×40 and 128	20, 20 and 3 KB
Decision-level fusion (small)	IMU backbone and audio backbone	200×6 and 198×40	20 and 20 KB

Table 4.8: Firmware variants for on-device inference. The tensor arena is the fixed memory block that holds the intermediate results of a network.

4.8.2 Model Conversion and Integer Quantization

The trained Keras models store their weights as 32-bit floating-point numbers. For the microcontroller they are converted into the TensorFlow Lite format and quantized to 8-bit integers [CITATION]. In this form, each value is represented by an

integer q between -128 and 127, and the original real value r is recovered as

$$r = s \cdot (q - z), \tag{4.3}$$

where the scale s and the zero point z are fixed for each tensor. Quantization reduces the size of the weights by a factor of four. It also allows all computations to be carried out in integer arithmetic, which is the form best supported by TensorFlow Lite Micro on microcontrollers.

Full integer quantization was used, so that not only the weights but also the intermediate results and the input and output of each network are 8-bit integers. To choose suitable scales for the intermediate results, the converter needs to know which value ranges occur in practice. It therefore receives a representative dataset of 200 standardised windows from the training split, which it passes through the network to record the range of every intermediate tensor. The validation data are not used in this step.

The conversion is carried out by an export script. Besides the converted model, it writes all values that the firmware needs to reproduce the preprocessing into C header files, which are listed in Table 4.9. The firmware is then compiled together with these files. Because all constants are produced automatically from the same objects that were used during training, errors caused by copying values by hand are ruled out. Figure 4.5 shows this process.

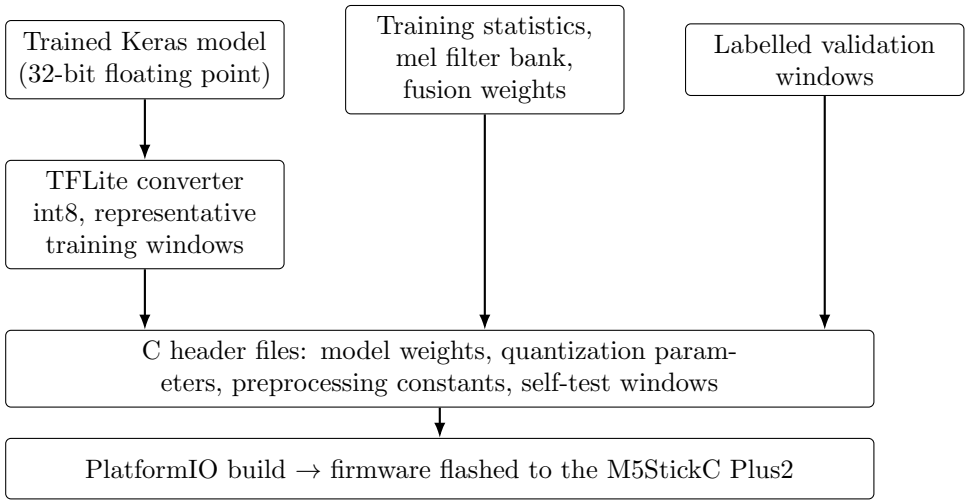


Figure 4.5: Export of a trained model to the device firmware. All constants used on the device are generated automatically from the training pipeline.

4.8.3 On-Device Signal Processing

The preprocessing described in Chapter 3 and Section 4.2 was originally implemented in Python and applied offline to complete recordings. On the device, the

Content	Source	Purpose
Mean and standard deviation of each input channel	training split	standardisation of the live input
Scale and zero point of input and output	TFLite converter	quantization of the input, conversion of the output back to real values
Mel filter bank (257×40)	TensorFlow, as in training	computation of the log-mel spectrogram
Hann window (400 values)	TensorFlow, as in training	weighting of each STFT frame
Class names and fusion weights	training	output and decision-level fusion
One labelled window per class	validation split	self-test at start-up

Table 4.9: Values embedded in the firmware in addition to the model weights.

same steps have to be carried out on a single two-second window, in real time and with little memory.

Inertial data. During data collection, the IMU delivered about 250 samples per second, which were reduced to 80 Hz by averaging consecutive samples, as described in Section 3.6.2. The firmware repeats this step. It collects all IMU samples that arrive during the two-second window and averages them in 160 consecutive groups, which corresponds to the 160 samples of a training window. The result is then resampled to 200 time steps by linear interpolation, standardised with the stored training statistics and quantized with the input scale of the network. To save memory, the last three steps are combined into a single pass that writes the result directly into the input tensor of the network.

Audio data. A two-second audio window consists of 32 000 samples. Stored as 16-bit integers, it would occupy 64 KB, which would not fit into memory together with the networks. The log-mel spectrogram is therefore computed while the audio is being recorded. The firmware reads the microphone in chunks of 160 samples, which corresponds to the 10 ms hop of the STFT, and keeps only the most recent 400 samples, which form the current frame. For each new frame, it multiplies the samples by the Hann window, computes the FFT with a length of 512, applies the mel filter bank and takes the logarithm. Only the resulting 198×40 mel values are kept. The samples are scaled to the range from -1 to 1 beforehand, as in the training pipeline. For the early fusion model, the spectrogram is then resampled to 200 frames and written into the audio channels of the fused input. The decision-level and embedding-level variants pass the 198×40 spectrogram to the audio network unchanged.

Two details of the training pipeline are easy to overlook and were reproduced explicitly. First, the STFT in TensorFlow applies a Hann window to each frame

by default, even though the training code does not request it. Second, the mel filter bank of TensorFlow differs slightly from those of other libraries. As listed in Table 4.9, both the window and the filter bank are therefore taken directly from TensorFlow and embedded as tables instead of being recomputed on the device.

4.8.4 Firmware Structure and Inference Loop

After start-up, the firmware initialises the display, the microphone and the networks and runs the self-test described in Section 4.8.5. It then enters a loop, shown in Figure 4.6, that repeats the following steps.

1. A two-second window is recorded. The IMU is read between successive audio chunks, so that both modalities cover the same period of time, as in the synchronised training data.
2. The inertial and acoustic inputs are prepared as described in Section 4.8.3.
3. The network or networks are executed.
4. For decision-level fusion, the output probabilities of both networks are converted back to real values with Equation 4.3 and combined with the per-class weights of Equation 4.2. For embedding-level fusion, the two embeddings are passed to the fusion head.
5. The predicted class and its confidence are shown on the display and sent over the serial port, together with the time required by each step.

Recording and inference take place one after the other. While a window is being processed, no new data are recorded, so consecutive windows do not overlap as they do in the training data. A new prediction is therefore available every two seconds plus the processing time.

All memory is reserved statically when the firmware is compiled. TensorFlow Lite Micro places all intermediate tensors of a network in a fixed memory block, the tensor arena, whose size has to be chosen in advance. The sizes in Table 4.8 were chosen so that all buffers fit into the available memory together, and the firmware reports at start-up how much of each arena is actually used. The network weights are not part of the arena, because they remain in flash memory and are read from there during inference. In addition, each network registers only the nine operations it actually uses instead of all available operations, which keeps the program smaller.

4.8.5 Verification of the Deployment Pipeline

When the device produces a wrong prediction, the cause can lie in two places. Either the network itself does not run correctly on the device, or the input it receives differs from the training data. To separate these two cases, each firmware runs a self-test

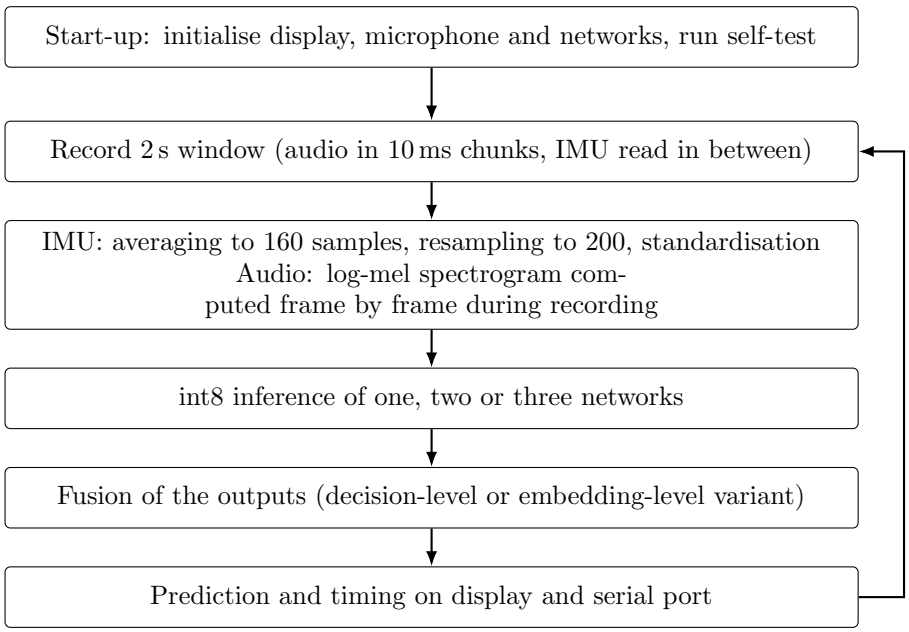


Figure 4.6: Structure of the inference firmware. After the start-up phase, recording, preprocessing, inference and output are repeated continuously.

at start-up. It uses one labelled window per class from the validation split. These windows were standardised and quantized in Python with the same constants as on the device and are embedded in the firmware. They are written directly into the input tensor, bypassing the sensors and the on-device preprocessing, and the device reports how many of them are classified correctly. If the self-test succeeds but live predictions are poor, the error must lie in the capture or in the preprocessing.

For the live pipeline, the firmware also prints diagnostic values in every cycle. These include the number of captured IMU samples, the mean acceleration along the three axes and the range of the raw audio samples. The mean acceleration shows how the device is oriented relative to gravity and can be compared with the training data. This makes it possible to detect problems such as a different wearing orientation, whose effect is examined in Chapter 5.

4.9 Evaluation Protocol

4.9.1 Grouped Train/Validation Split

As described in Section 3.7, the windows were created with 50 % overlap, so neighbouring windows share half of their samples. A random split would place such neighbours on both sides of the train/validation boundary. The model would then be evaluated on data that it had partly seen during training, and the measured accuracy would overestimate its performance on new data [CITATION].

To prevent this, the windows are first grouped into activity instances. Consecutive

windows with the same label belong to the same group as long as their start times are at most 1.5 s apart, that is, 1.5 times the window stride. Consecutive windows of one uninterrupted activity occurrence start exactly 1 s apart, so the additional 0.5 s serves as a margin. Small irregularities in the window start times therefore do not split one occurrence into several groups, while occurrences separated by a pause or by another activity, which leave a gap of at least one further stride, are still kept apart. The 2565 windows form 125 such groups. The split is then made separately for each class. The groups of a class are shuffled, and whole groups are assigned to the validation set until it holds at least 20 % of that class's windows. The remaining groups form the training set. Every activity instance therefore lies entirely in one of the two sets, and a fixed random seed makes the split identical for all models, so that their results can be compared directly. Figure 4.7 illustrates the principle.

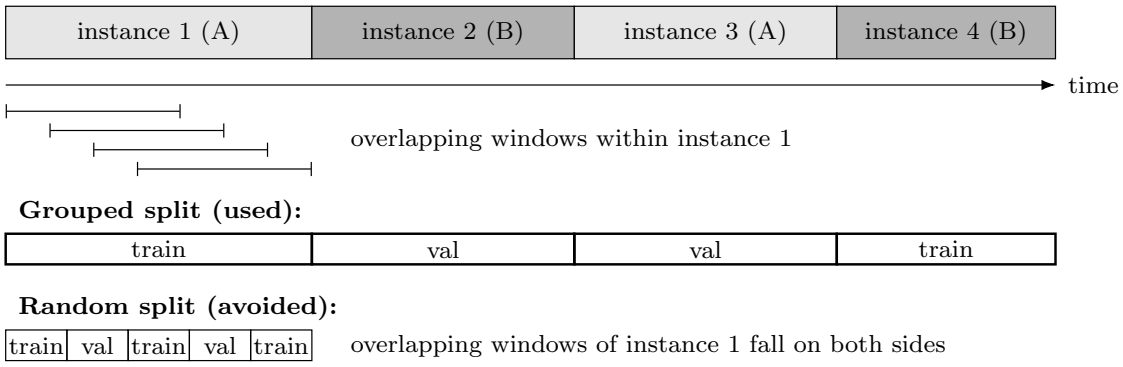


Figure 4.7: Grouped splitting. Overlapping windows are grouped into activity instances, and whole instances are assigned to either the training or the validation set. A random split would distribute the overlapping windows of one instance across both sets.

Table 4.10 shows the resulting split. Because only whole groups can be moved, the validation share of a class is not exactly 20 %. It exceeds the target considerably when a class consists of a few long instances, as is the case for `walk`, `measuring` and `place_container`, where a single group already contains a large part of the class. Overall, 946 windows (37 %) fall into the validation set and 1619 windows (63 %) into the training set. For `place_container`, only 23 windows remain for training. This uneven split is a consequence of keeping activity instances intact, and its effect on the results is discussed in Chapter 6.

4.9.2 Metrics

Overall accuracy, the share of correctly classified validation windows, is used as the main measure for comparing the models. Because the classes are imbalanced, accuracy alone can hide poor performance on small classes. Precision, recall and F1-score are therefore also reported for each class, together with the macro-averaged F1-score, which gives every class the same weight [CITATION]. Confusion matrices

Class	Training	Validation	Total	Validation share
assemble_parts	130	36	166	22 %
carry_container	175	45	220	20 %
hammering	170	80	250	32 %
idle	187	120	307	39 %
insert_screws	185	61	246	25 %
measuring	159	161	320	50 %
place_container	23	51	74	69 %
screw_tightening	454	142	596	24 %
walk	136	250	386	65 %
Total	**1619**	**946**	**2565**	**37 %**

Table 4.10: Training and validation windows per class after the grouped split.

are used to identify which activities are mistaken for one another. Unless stated otherwise, all results are reported on the validation set of the grouped split and come from a single training run per model.

4.9.3 On-Device Measurements

The deployed models are evaluated in three ways. First, the accuracy of the quantized models is measured in Python on the full validation set and compared with that of the original floating-point models, which isolates the effect of quantization. Second, the firmware reports the size of each model, the used part of each tensor arena and the time needed for capture, preprocessing and inference, measured with the system timer of the microcontroller. Third, the self-test and live tests with the device worn on the wrist show whether the complete pipeline works under real conditions.

4.10 Chapter Summary

This chapter introduced the models compared in this thesis. Inertial windows are processed as 200×6 time series by a 1D CNN, and audio windows as 198×40 log-mel spectrograms by a 2D CNN, for which three pooling variants were developed to make use of the temporal structure of the sounds. The two modalities are combined at the input level (channel concatenation and unified spectrogram), at the level of learned embeddings, and at the level of class probabilities, where a per-class weighting reflects the different reliability of the two modalities across activities. A random forest on hand-crafted features serves as a classical baseline, and reduced, regularised variants prepare the deployment on the device. All models share the same input normalisation, class weighting, training settings, best-epoch selection and grouped train/validation split. Selected small models are converted to 8-bit

integer arithmetic and executed on the wearable device by firmware that reproduces the complete preprocessing of the training pipeline and checks its own correctness with embedded validation windows. Chapter 5 presents the results of both the offline and the on-device evaluation.

5 Results and Evaluation

6 Discussion

7 Conclusion